

Modern Software Architecture, Containerization, and Kubernetes

INTRODUCTION TO KUBERNETES

Frank Heilmann

Platform Architect and Freelance
Instructor



About Me

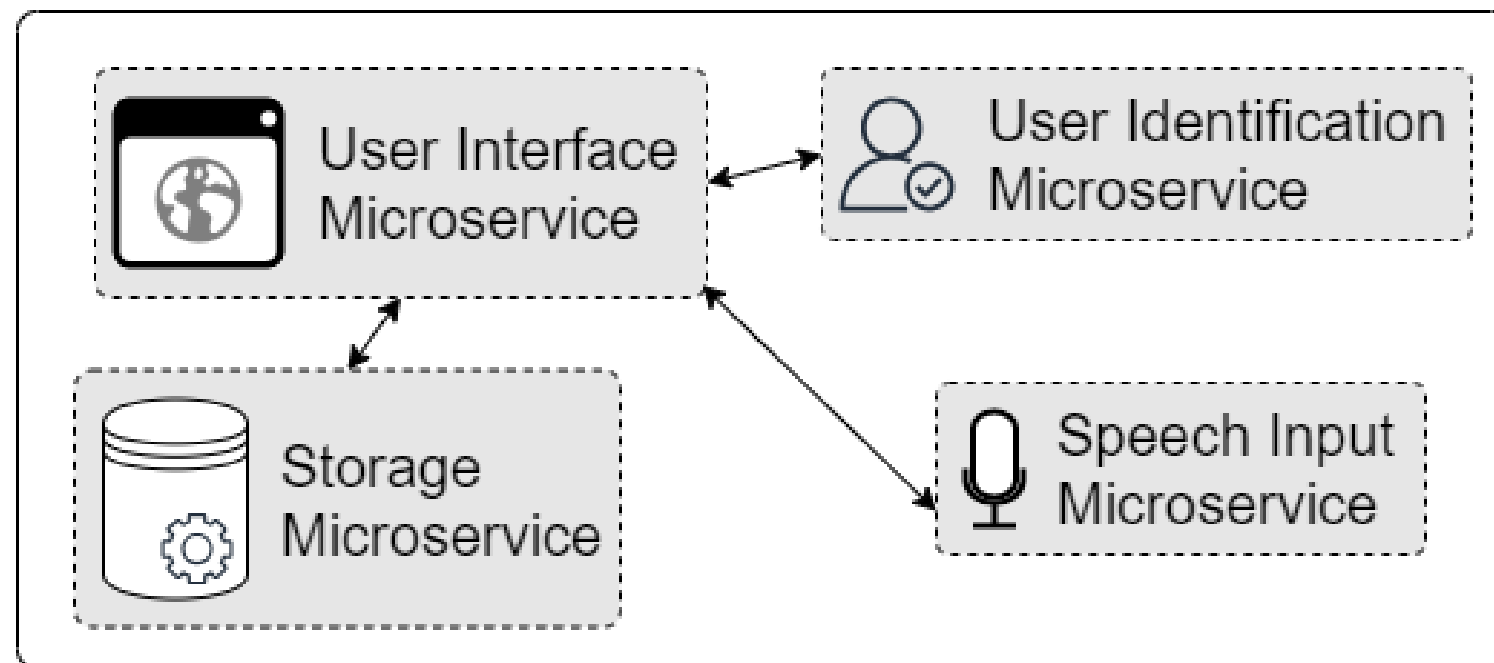
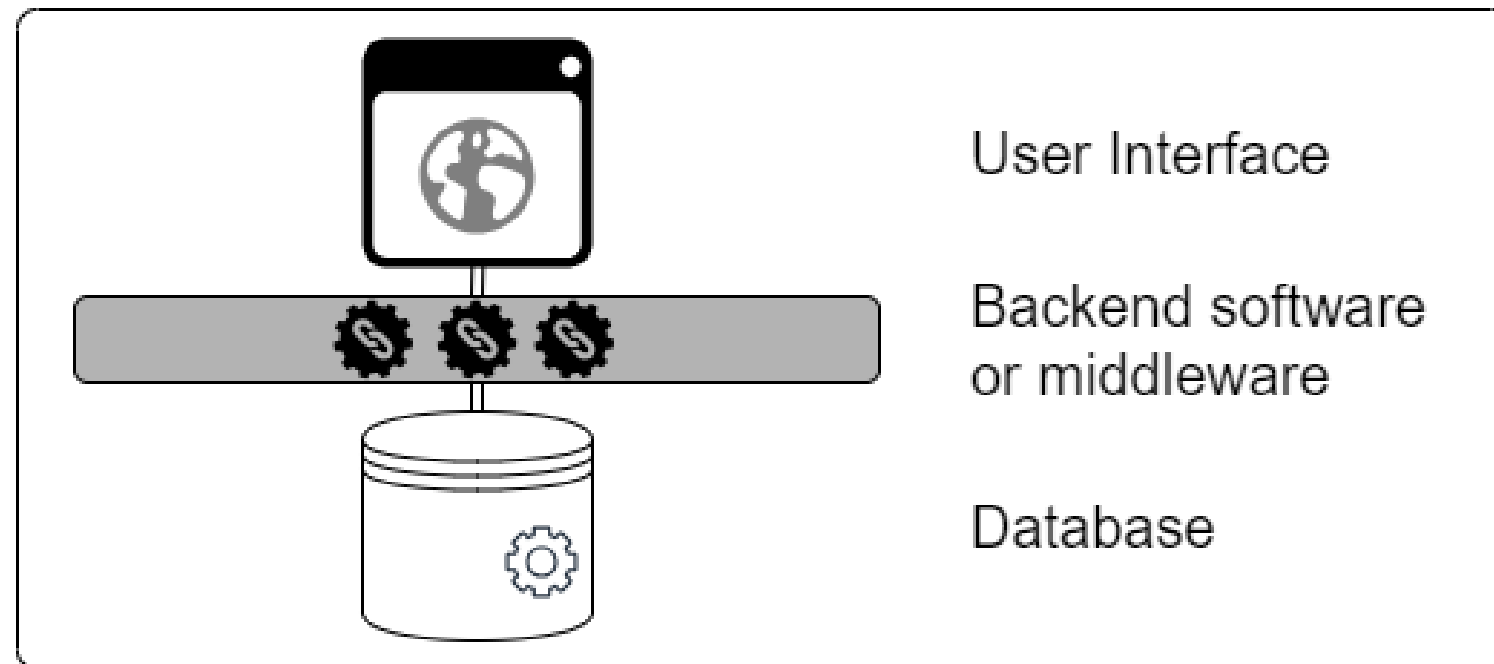


- Experienced Platform Architect, projects in different sectors
- Into Kubernetes since 2018 with focus on scalable data platforms on-prem and using major cloud providers
- Focus on resilience and high availability



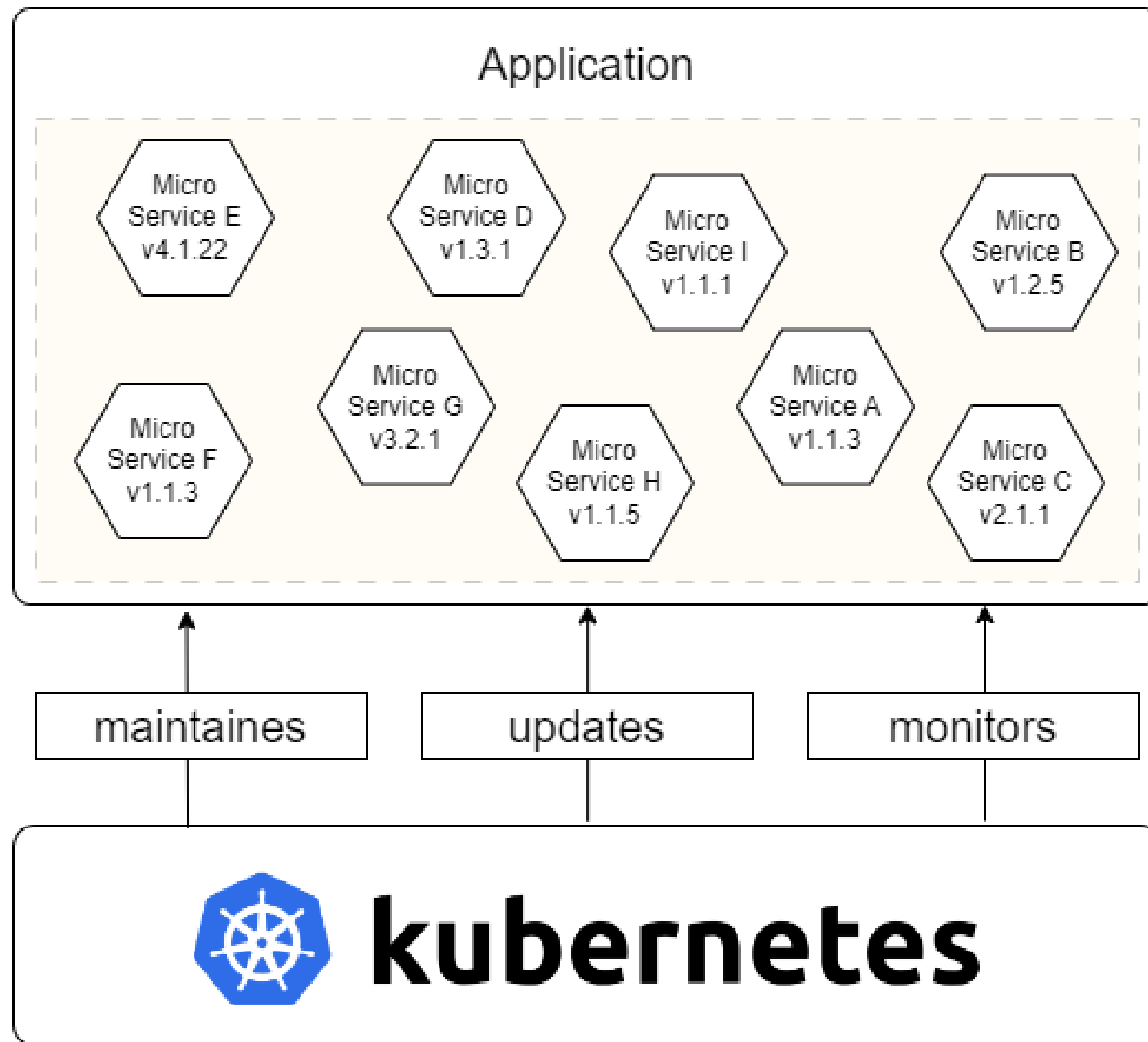
¹ <https://de.linkedin.com/in/frank-heilmann-19556590>

Modern Software Architecture



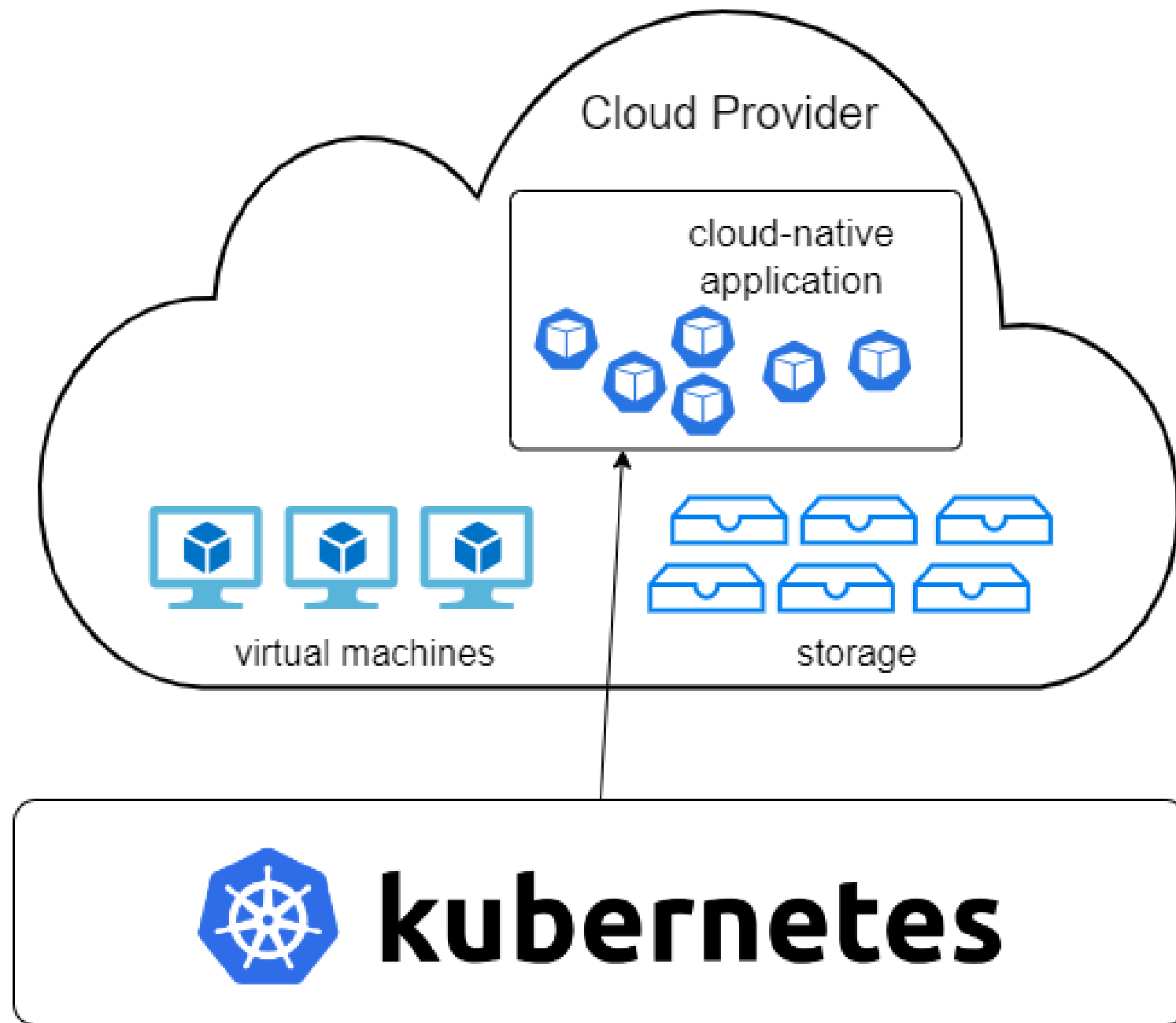
- Traditional architecture: **monoliths**
- Single applications, typically run independent from other applications
 - Hard to maintain and to update
- Modern architecture: constructed from independent building blocks: **microservices**
 - Can be independently maintained and updated
- Ideally suited for cloud computing

Containerization and Kubernetes



- Modern applications consist of potentially thousands building blocks
- Deployed via **containers!**
 - Each building block is *delivered* in an individual container
- **Kubernetes** keeps track of all containers

Cloud Nativeness and Kubernetes



- *Cloud-native*: a way to build & deploy applications in the cloud
- **Cloud-native applications** are designed to be scalable
- **Kubernetes** is cloud-native:
 - simplifies deploying and managing containers
 - enables easy scaling of applications

Kubernetes Distributions and Cloud Offerings

- Kubernetes is an open-source project by Google
- Everyone can download, install, and use it
- We can also use Kubernetes with all cloud providers as a managed service
- "Kubernetes" --> "K *ubernete* s" --> "K8s"

Let's practice!
INTRODUCTION TO KUBERNETES

Docker and Kubernetes

INTRODUCTION TO KUBERNETES



Frank Heilmann

Platform Architect and Freelance
Instructor

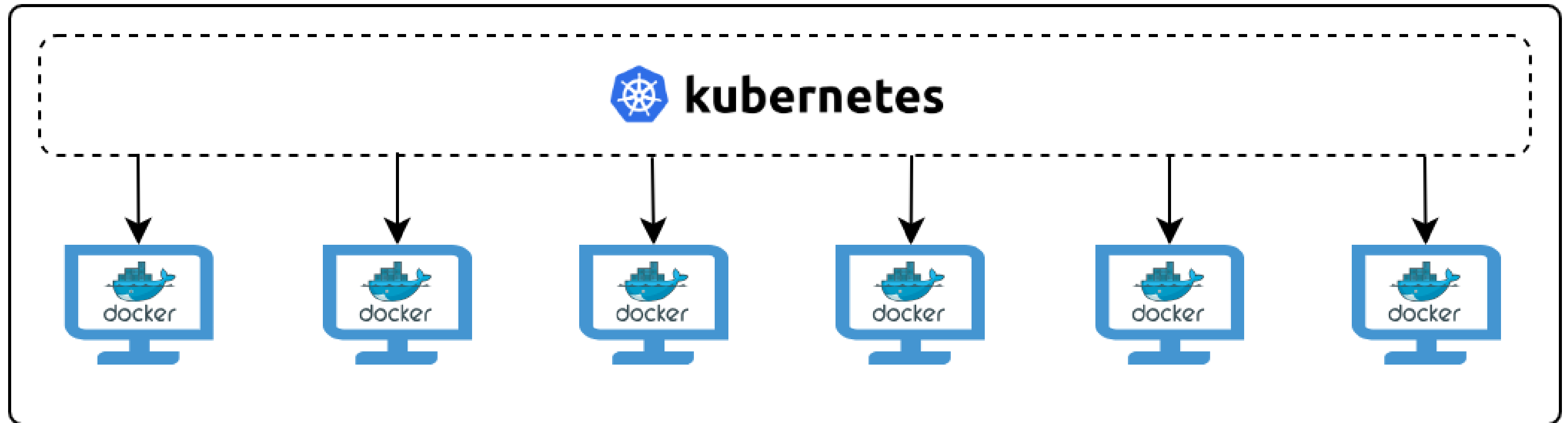
Container Orchestration Tools



- Modern software stacks typically consist of potentially thousands of individual containers
- Managing all these containers is known as **Container Orchestration**, several **container orchestration tools** exist
- Kubernetes has an estimated market share well above 95%

Kubernetes for Orchestration

- Kubernetes solves the typical challenges of container orchestration, e.g.,
 - scheduling and networking (where to deploy a container and how to connect them)
 - how to attach storage to a container
- To do that, Kubernetes interacts with **Container Engines**.



The Relationship between Docker and Kubernetes

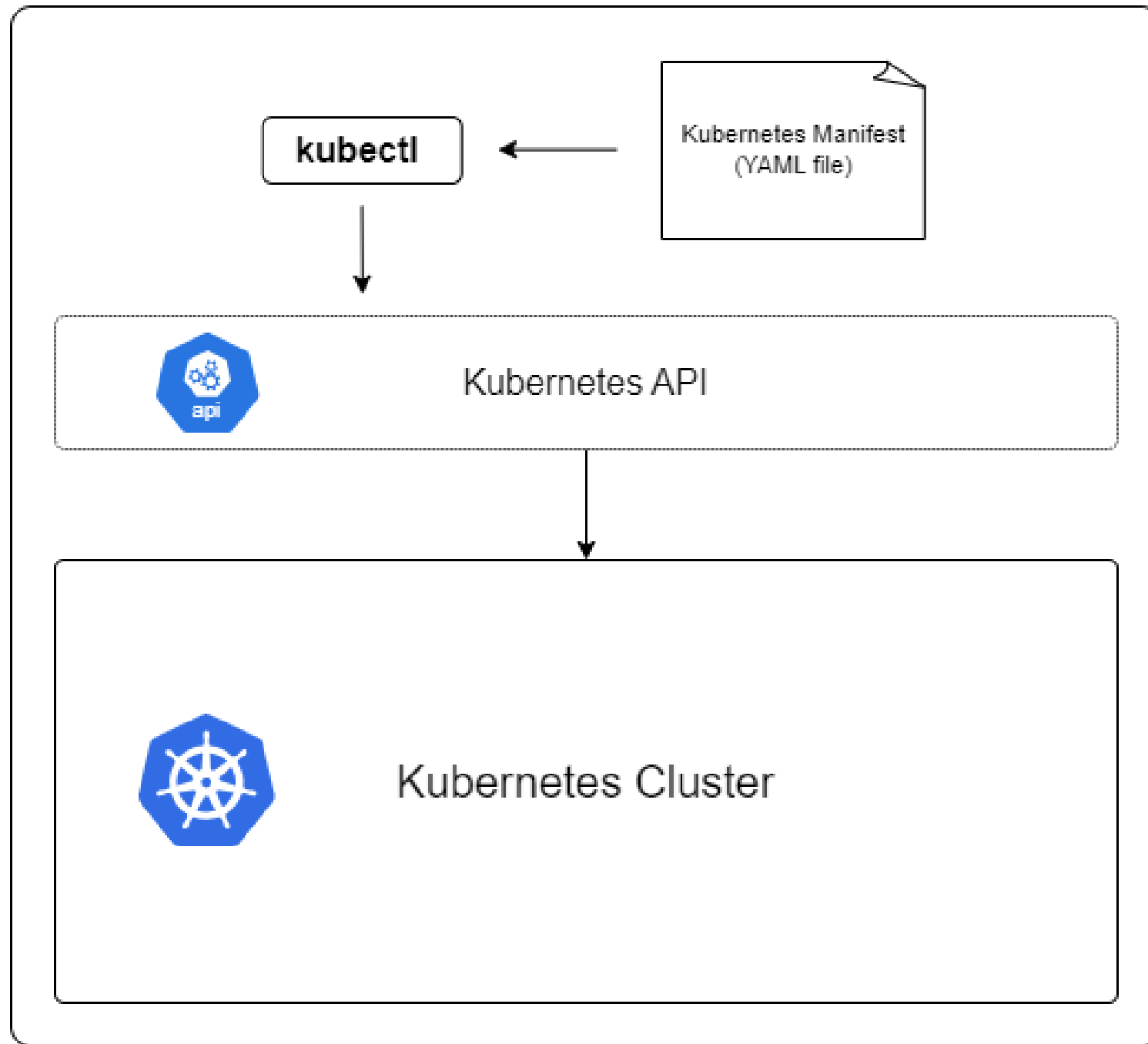
- Often, **Docker** is your container engine of choice
- Kubernetes interacts with Docker as a container engine to schedule and maintain containers
- Docker is typically used for two tasks:
 - creating and updating Docker images
 - starting containers from such images
- Kubernetes never creates Docker images, you use Docker for this

Kubernetes Manifests

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25.4
          ports:
            - containerPort: 80
```

- Kubernetes objects (e.g., containers) are described in so-called **Kubernetes Manifests**
- Manifests are **YAML files** that describe which objects you want, how they should be configured, where they should be scheduled, and a lot more
- Manifest are **declarative**, i.e., you describe **what** you want, or which **state** to achieve
- They are **not imperative**, you do **not describe how to achieve it**

kubectl



- `kubectl` is a command line tool to interact with Kubernetes
- `kubectl` comes with many commands and options
- `kubectl` reads your Manifest, sends them to Kubernetes via its API, and Kubernetes will compute what to do to achieve the state you want
- Pronounced *cube cuddle* ;-)

Let's practice!
INTRODUCTION TO KUBERNETES

Kubernetes Architecture

INTRODUCTION TO KUBERNETES



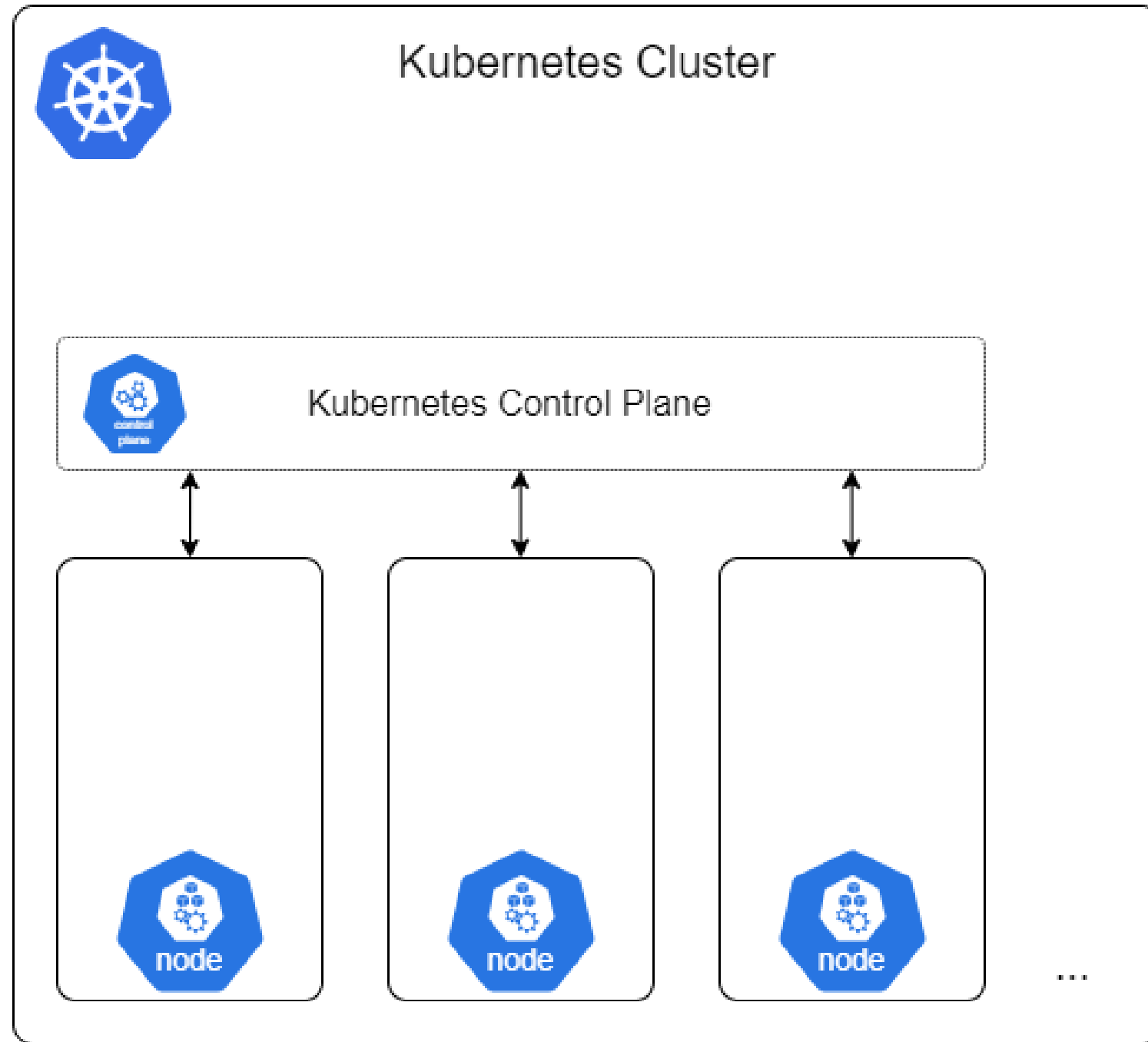
Frank Heilmann

Platform Architect and Freelance
Instructor

Kubernetes Overview

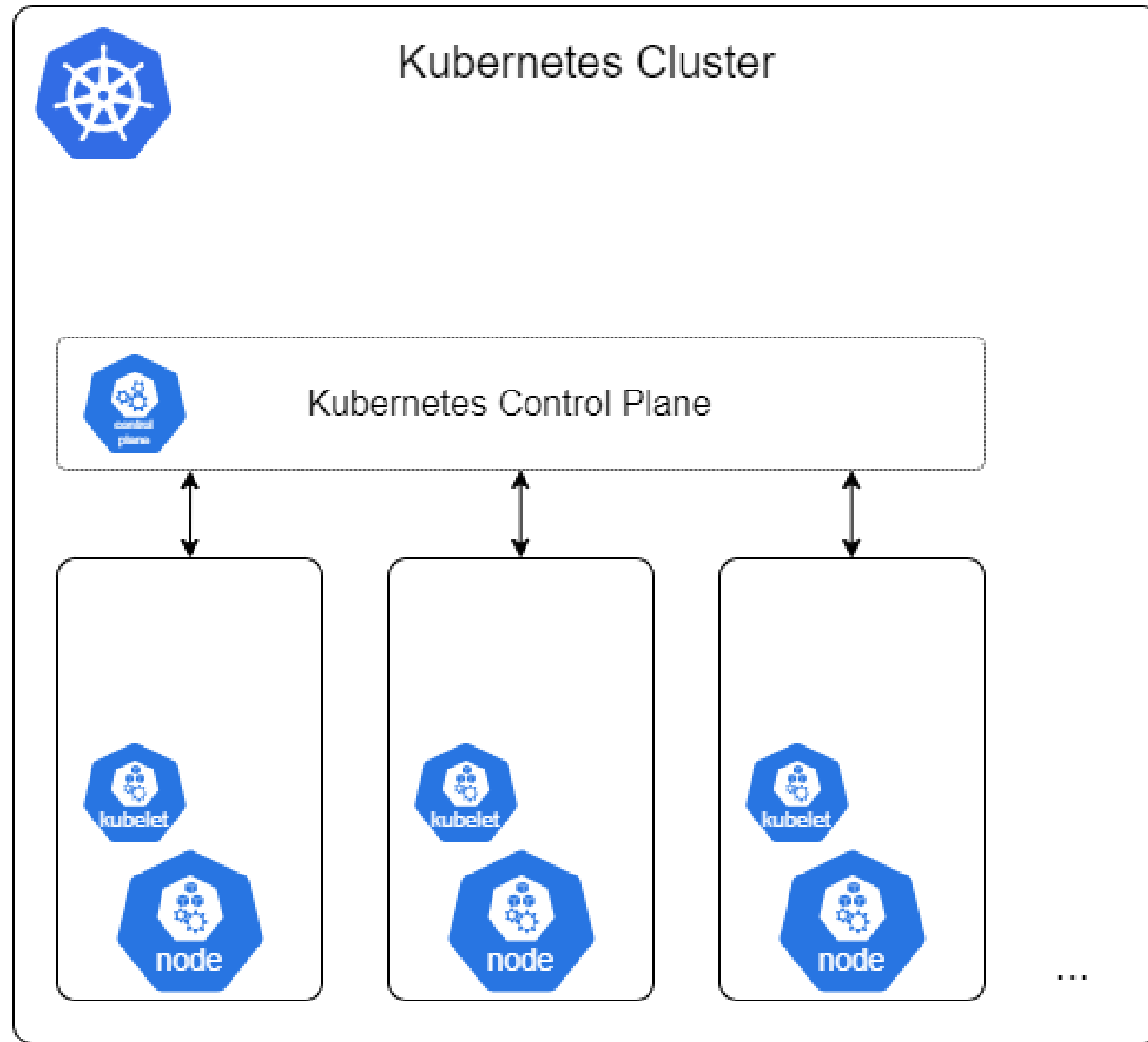
- Kubernetes is built from many elements:
- Most important ones, from larger to smaller:
 - **Clusters and Control Planes**
 - **Nodes**
 - **Pods**
- Network connectivity through **Services**

Kubernetes Cluster and the Control Plane



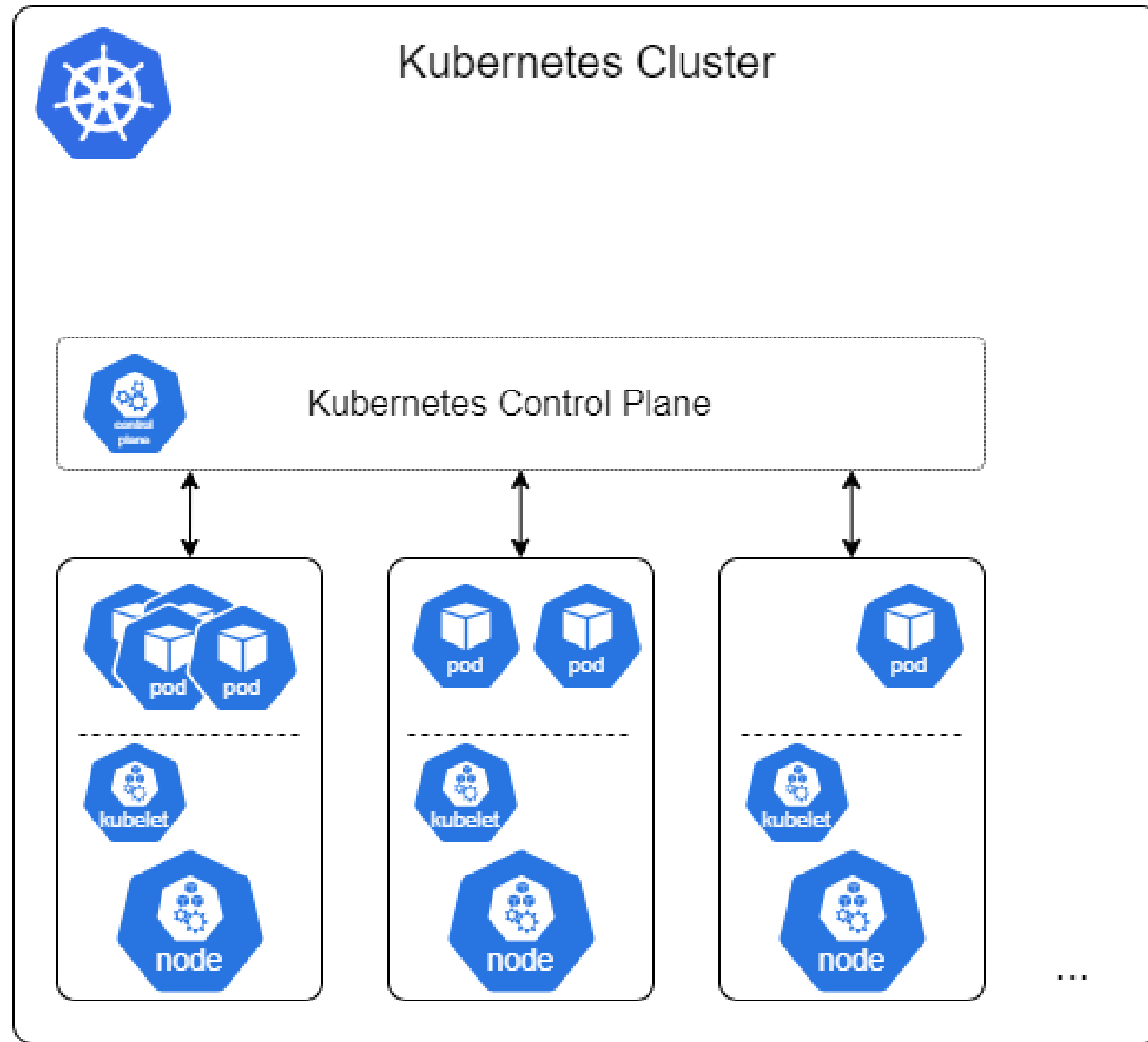
- A Kubernetes Cluster is a set of connected computers (or **Nodes**)
- Servers in a datacenter, virtual machines in the cloud
- The Kubernetes **Control Plane** manages these nodes
 - consists of many components, that can run on any node in the cluster

Kubernetes Nodes



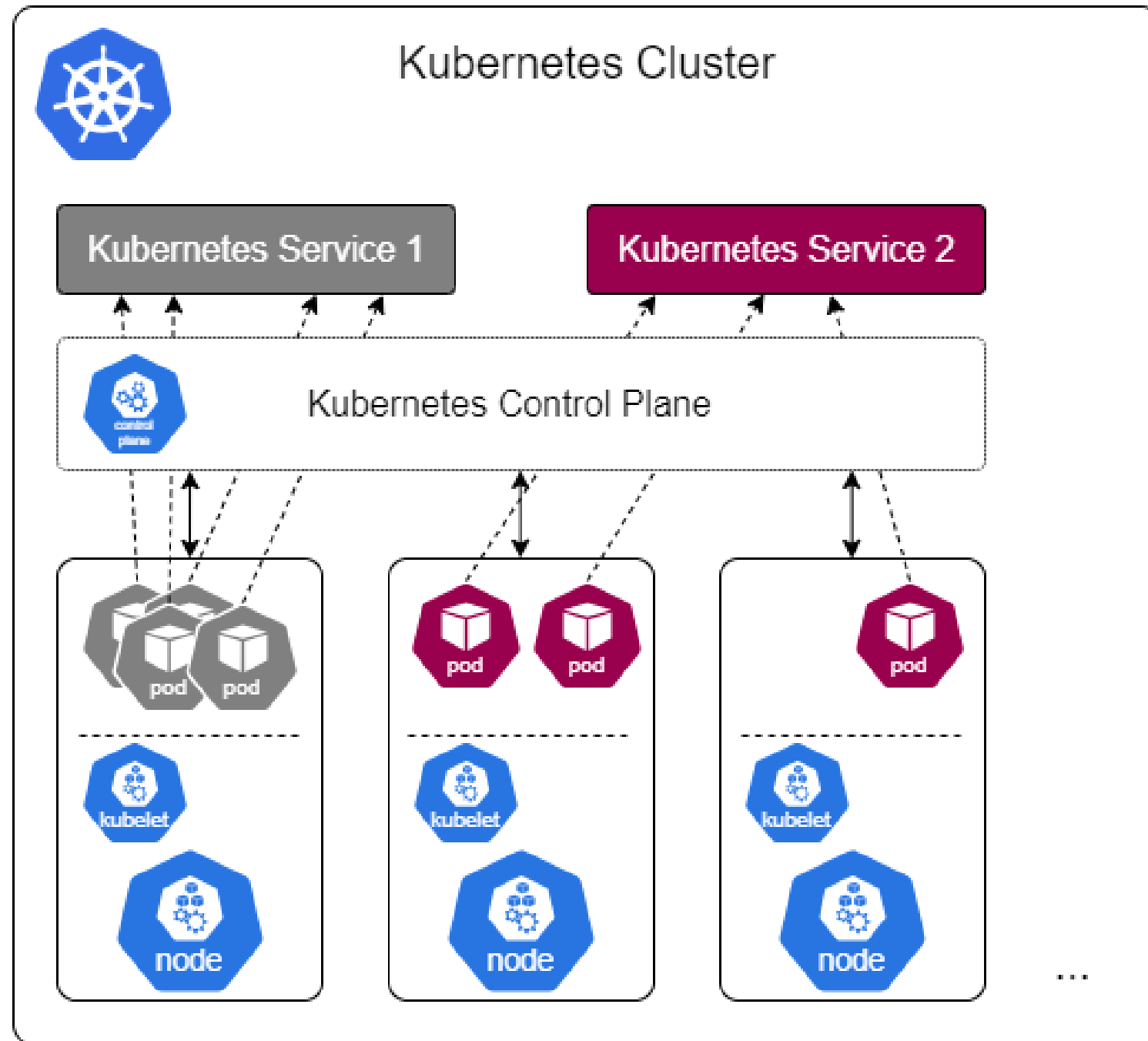
- A Kubernetes Node typically runs Linux + container engine (Docker)
- Nodes are also called **worker machines**
- Nodes run **Kubernetes Kubelet**
 - ensures containers run in so-called *Pods*

Kubernetes Pods



- Kubernetes **Pod**
 - Smallest unit that you can deploy
 - A Pod is a set of one or more containers
- The containers in a Pod belong together logically, share storage and network
- Pods are *ephemeral*:
 - Pods can be stopped and recreated and any point in time.
 - Pods can moved to other nodes at any point in time.

Kubernetes Services



- **Kubernetes Service:** resource for exposing network connectivity
- Required to connect to Pod from outside, or to communicate between Pods
- Reason: Pods may get re-deployed any time, and will
 - Receive a new IP address
- Services are not *ephemeral*, they offer stable network connectivity

Kubernetes Cheat Sheet

- **Kubernetes Cluster:** set of connected computers (Nodes) configured to run Kubernetes
- **Kubernetes Control Plane:** manages the Nodes in a Cluster
- **Kubernetes Nodes:** also called "worker machines", running Linux and a container engine
- **Kubernetes Pods:** a set of one or more containers, the smallest deployable unit
- **Kubernetes Services:** a resource for exposing network connectivity, required to connect to Pods from outside, and for communication between Pods

Let's practice!
INTRODUCTION TO KUBERNETES

Deploying a First (Stateless) Application

INTRODUCTION TO KUBERNETES



Frank Heilmann

Platform Architect and Freelance
Instructor

More on "kubectl"

- `kubectl` : main command to interact with Kubernetes objects
- Objects are, e.g., `pod` , `service` , etc.
- Typical usage patterns:
 - `kubectl create -f <Manifest.yml>` : create new objects, with `-f` for "filename"
 - `kubectl apply -f <Manifest.yml>` : create new objects & change the state of objects
 - `kubectl get <object>` : overview about objects deployed on Kubernetes
 - `kubectl describe <object>` : detailed information about an object
- Detailed help available via command line option `--help`

More on Manifests

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 5
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25.4
          ports:
            - containerPort: 80
```

- Remember: Manifests are **declarative**
- Typically YAML, but also in JSON format
- Two important sections:
 - **metadata**: essential information about the object or resource
 - **spec**: defines the specifications, or desired state, of the object or resource
- Sections can be quite deep, depending on the resource to be deployed

Stateless Applications

- Stateless apps:
 - General concept
 - Not specific to Kubernetes
 - Do not save an internal state, or context of processed data
- When interrupted, a new replica of the stateless app is recreated and starts operating.
- Examples:
 - The database frontend querying a database backend
 - A search app querying a full text index
 - A data stream app that converts temperature readings from an IoT sensor from °F to °C

Kubernetes Deployments

- "Stateless applications" translate to "Kubernetes Deployments"
- A sample Manifest consists of:
 - `apiVersion` and `kind`
 - `metadata` and `spec`
- `spec` defines number of `replicas`, a `selector`, and a `template`
- More on `selector` later
- `template` describes details for the creation the pods in the Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: <deployment name>
  labels:
    app: <a label for the application>
spec:
  replicas: <number of initial replicas>
  selector:
    matchLabels:
      app: <matches the label above>
  template:
    metadata:
      labels:
        app: <label to be given to each pod>
    spec:
      containers:
        - name: <container name>
          image: <the image to be used>
          ports:
            - containerPort: <ports for networking>
```

Deploying to a Kubernetes Cluster

- `kubectl apply -f <manifest.yaml>` for creating pods and applying changes.
- Kubernetes Control Plane will schedule the Deployment on Nodes.
 - Then, Pods created is triggered on the Nodes.
- Pods get a unique, but random (unpredictable) identifier, each Pod is "as good as any other"

Let's practice!
INTRODUCTION TO KUBERNETES

Scaling and Monitoring an Application

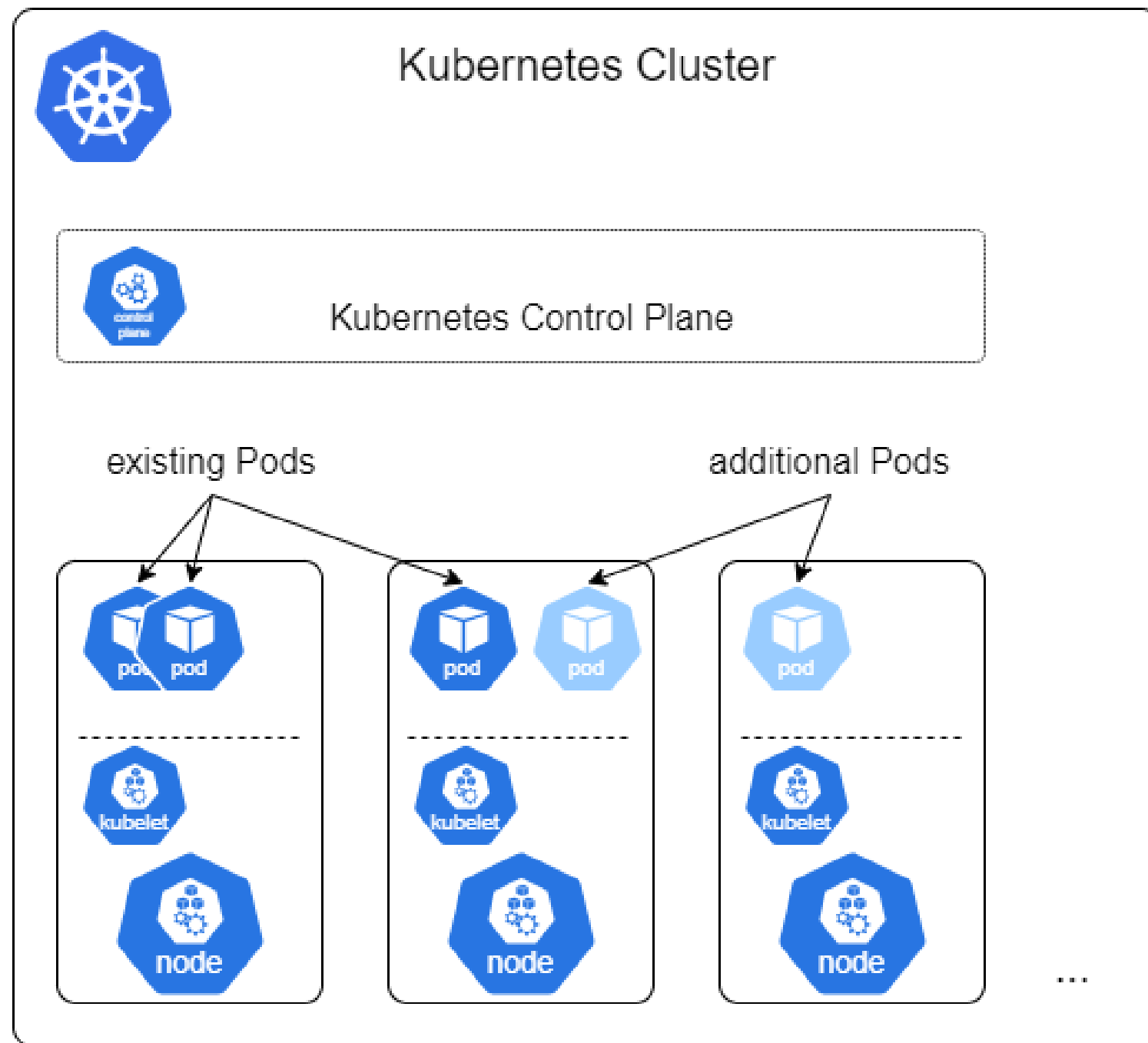
INTRODUCTION TO KUBERNETES



Frank Heilmann

Platform Architect and Freelance
Instructor

Scaling on Kubernetes



- Scaling is a technique to add (scale up) or remove (scale down) resources:
 - Scale up: react to increasing load
 - Scale down: save resources
- Scaling the number of Pods is easy:
 - Either change the number of `replicas` in the Manifest and re-apply,
 - Or use the command `kubectl scale deployment ...`
 - with `--replicas <number>`

Scalability and Cloud Nativeness

- An application needs to be designed for scalability
- Legacy applications, in particular monoliths, are typically not scalable in the way shown here
- Modern, cloud native applications are designed with the the goal to be easily scalable

Monitoring an Application

- Monitoring: observing applications in real-time
 - Enables reaction to all kind of problems
- Examples of modern monitoring application for Kubernetes:
 - Prometheus, Grafana, or `kubectl`
- Here, we use `kubectl` for basic monitoring tasks
- Typical command:
`kubectl get <object to be monitored>`
- **Example 1:** `kubectl get pods` returns all pods
- **Example 2:** `kubectl get services` returns all services

Let's practice!
INTRODUCTION TO KUBERNETES

Deploying, Scaling, and Monitoring a Stateful Application

INTRODUCTION TO KUBERNETES



Frank Heilmann

Platform Architect and Freelance
Instructor

Recap Stateless Applications

- Short recap: stateless applications map to "Deployments" in Kubernetes
- Used when each Pod of the application has exactly the same tasks
- Stateful applications need Pods that belong together in set, but may work on different tasks and different data
- Much of what we have learned about Deployments can be applied to StatefulSets as well

Stateful Applications

- Stateful apps:
 - general concept
 - fit well to Kubernetes
 - save some state
- When interrupted or stopped, a new replica (Pod) can read the saved state and continue operating from this state
- Example:
 - A database backend (e.g. PostgreSQL) delivers data to a frontend using 3 Pods.
 - Each time we update data using any of the Pods, that data needs to be persisted
 - When a Pod terminates, a new one is created and needs to pick up the saved state

Kubernetes StatefulSets

- Stateful applications translate to "Kubernetes StatefulSets"
- A sample manifest consists of the same sections like:
 - `apiVersion`, `kind`, `metadata`, `spec`, `template`
- `replicas` defines the number of Pods in the `StatefulSet`
- More on `selector` later

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: <deployment name>
  labels:
    app: <a label for the application>
spec:
  replicas: <number of initial replicas>
  selector:
    matchLabels:
      app: <matches the label above>
  template:
    metadata:
      labels:
        app: <label to be given to each pod>
    spec:
      containers:
        - name: <container name>
          image: <the image to be used>
          ports:
            - containerPort: <ports for networking>
```

Deploying to a Kubernetes Cluster

- `StatefulSet` is deployed similar to `Deployments` : `kubectl apply -f <manifest.yml>`
- Once deployed, a `StatefulSet` is created different than a `Deployment` :
 - Pods are created one after the other, not all at once like Pods in a `Deployment`
 - Pods get predictable names like `pod-0` , `pod-1` , `pod-2` . etc.
- This means: in contrast to the Pods of a `Deployment`, the Pods of a `StatefulSet` have an identity, and a state
- Hence, different Pods of a `StatefulSet` with different identity can perform different roles in an application

Scaling A StatefulSet

- Like Deployments, StatefulSets can be scaled up or scaled down:
 - Either change the number of `replicas` in the Manifest and re-apply,
 - Or use the command `kubectl scale statefulsets ...`
- When scaling up, new Pods will be created one after another:
 - e.g, `pod-0` , `pod-1` , `pod-2` first `pod-3` , then `pod-4` will be added
- When scaling down, Pods created last will be deleted first:
 - e.g, first `pod-4` , then `pod-3`

Monitoring a StatefulSet

- Like in the case of Deployments, Monitoring enables reactions to all kind of problems, like outages, load spikes, or missing storage
- Here, we use `kubectl` for basic monitoring tasks
- Typical command: same like with Deployments
- **Example 1:** `kubectl get pods` returns all pods in a StatefulSet with their current status
- **Example 2:** `kubectl get services` returns all services that a StatefulSet may use

Let's practice!
INTRODUCTION TO KUBERNETES

Deploying, Scaling, and Monitoring Kubernetes Storage

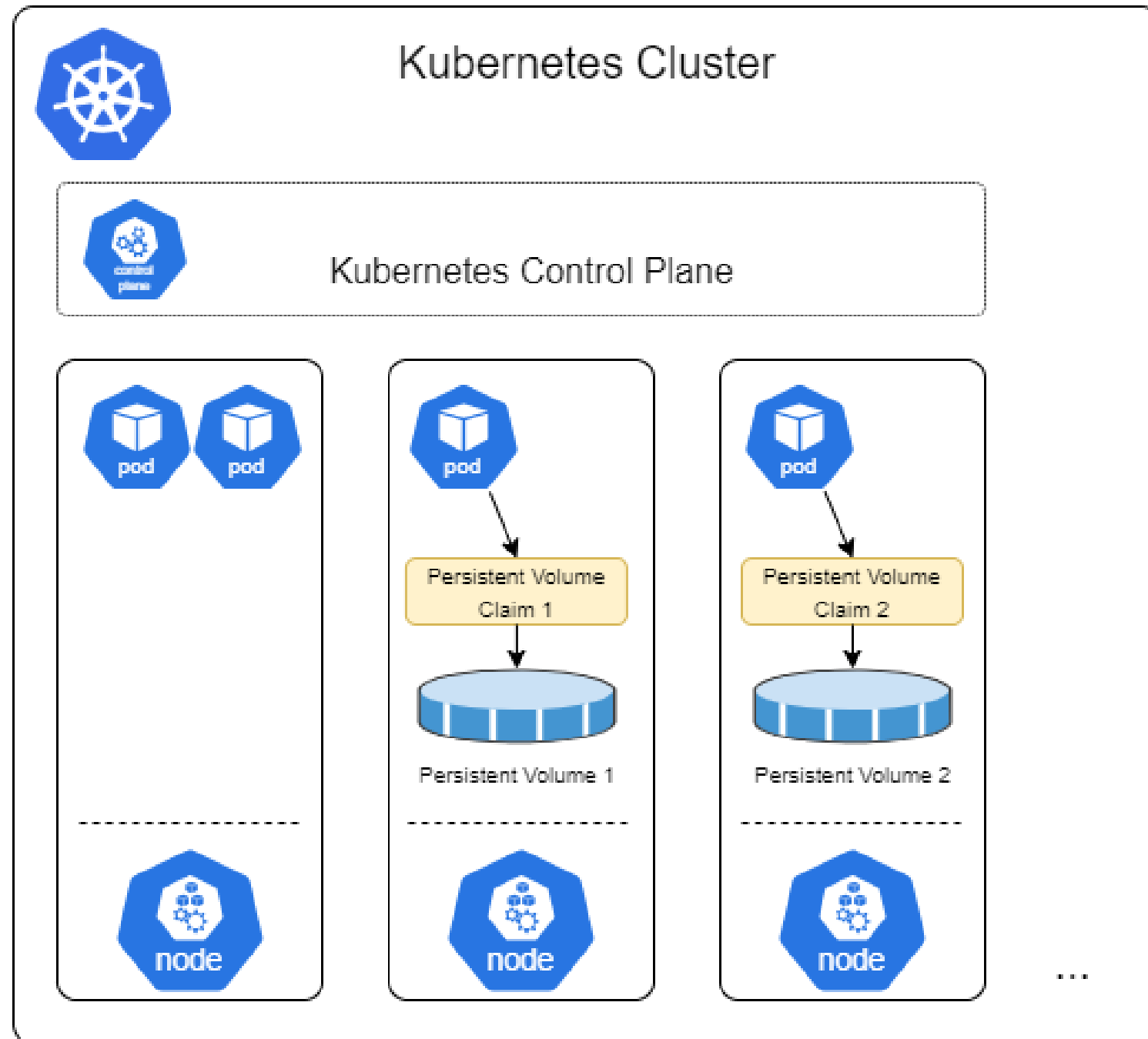
INTRODUCTION TO KUBERNETES



Frank Heilmann

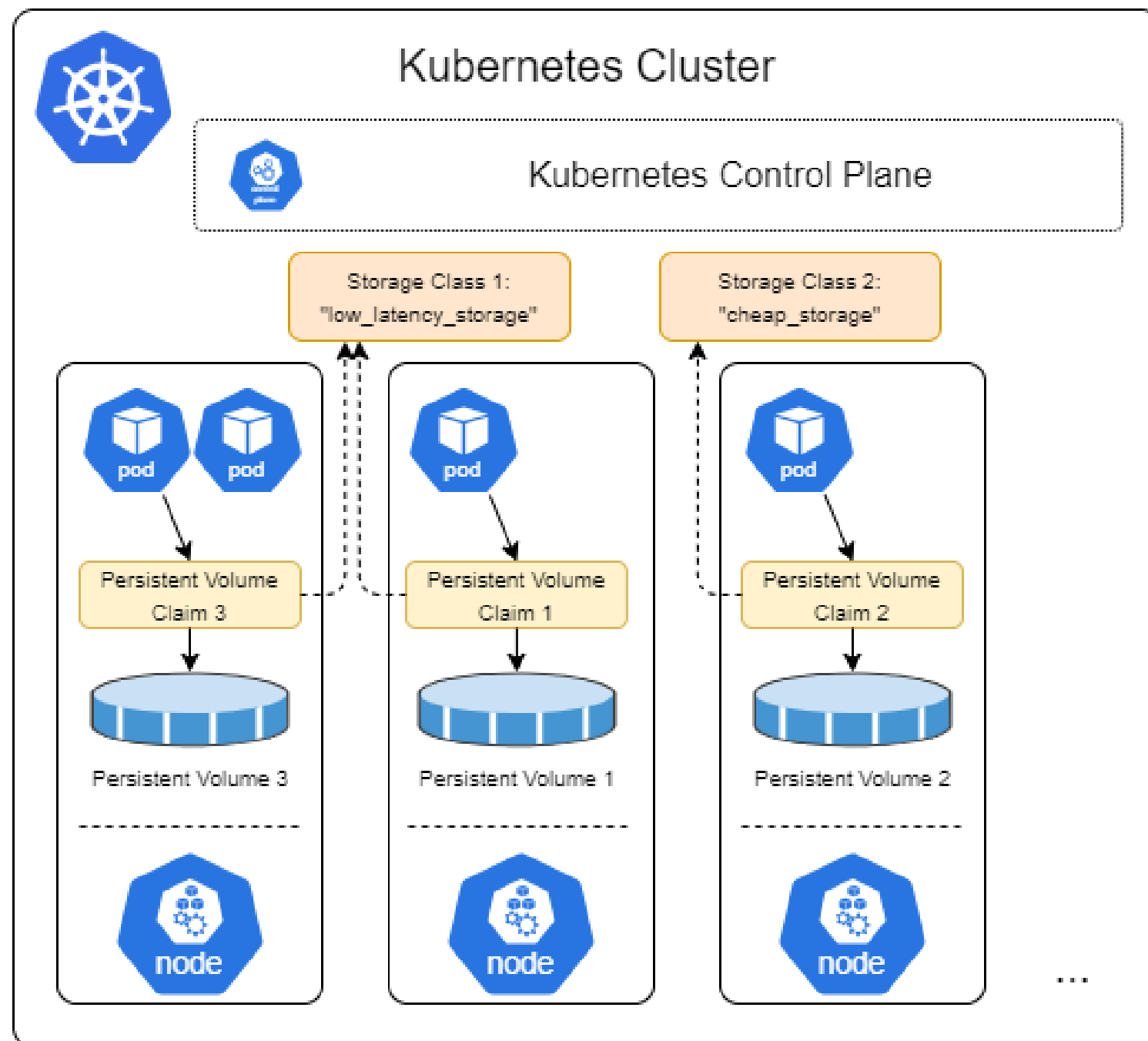
Platform Architect and Freelance
Instructor

Persistent Volumes and Persistent Volume Claims



- Fundamental Objects for storage: **Persistent Volumes (PV)**, maintained in parallel to Pods
- PVs are mapped to Pods using **Persistent Volume Claims (PVC)**
- A mapped PV allows data persistence when the Pod is stopped, killed, or restarted
- PVs enable the separation of storage and compute

Storage Classes



- PVs: provisioned either
 - manually by an Kubernetes admin
 - dynamically by regular user
- Dynamic provisioning happens via **Storage Classes (SC)** without human intervention
- **Storage Classes (SC):**
 - defined by Kubernetes admin
 - different types (different latency, e.g., SDD vs HDD, different backup strategies)
- If in doubt, use Storage Classes ;-)

Putting it all together

- There are only three objects that make storage work:
 - `PersistentVolume`
 - `PersistentVolumeClaim`
 - `StorageClass`
- A Pod with demand for persisted data uses a `PersistentVolumeClaim`
- This PVC has Kubernetes create a `PersistentVolume` for the Pod
- This `PersistentVolume` is mapped to the claiming Pod
- A named `StorageClass` is used, which defines details like latency and backup strategy of the PV
- This `PersistentVolume` survives (together with stored data), even when the `Pod` is terminated

Manifest Snippets

Pod with PersistentVolume

```
apiVersion: v1
kind: Pod
...
spec:
  containers:
    ...
    volumeMounts:
      - name: pv-mydata
        mountPath: /mydata
  volumes:
    - name: pv-mydata
      persistentVolumeClaim:
        claimName: datacamp-pvc
```

PersistentVolumeClaim with StorageClass

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: datacamp-pvc
spec:
  storageClassName: "standard"
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```


"kubectl" Commands For Storage

- `kubectl` offers a complete set of commands to create and monitor Kubernetes Storage
- Examples:
 - `kubectl get sc` lists all available Storage Classes
 - `kubectl get pvc` lists all deployed Persistent Volume Claims
 - `kubectl get pv` lists all deployed Persistent Volumes
 - As usual, `kubectl apply -f <manifest>` can be used to deploy storage resources that are declared in Manifests.

Let's practice!
INTRODUCTION TO KUBERNETES

Networking, Load Balancing, and Security

INTRODUCTION TO KUBERNETES



Frank Heilmann

Platform Architect and Freelance
Instructor

More on Labels and Selectors

Labels:

- Key/Value pairs attached to Kubernetes objects like Pods or Nodes
- Can be used to organize subsets of objects
- Can be modified at any time
- Examples:
 - `environment: prod`
 - `app: my_cool_app`
 - `has_GPU: true`

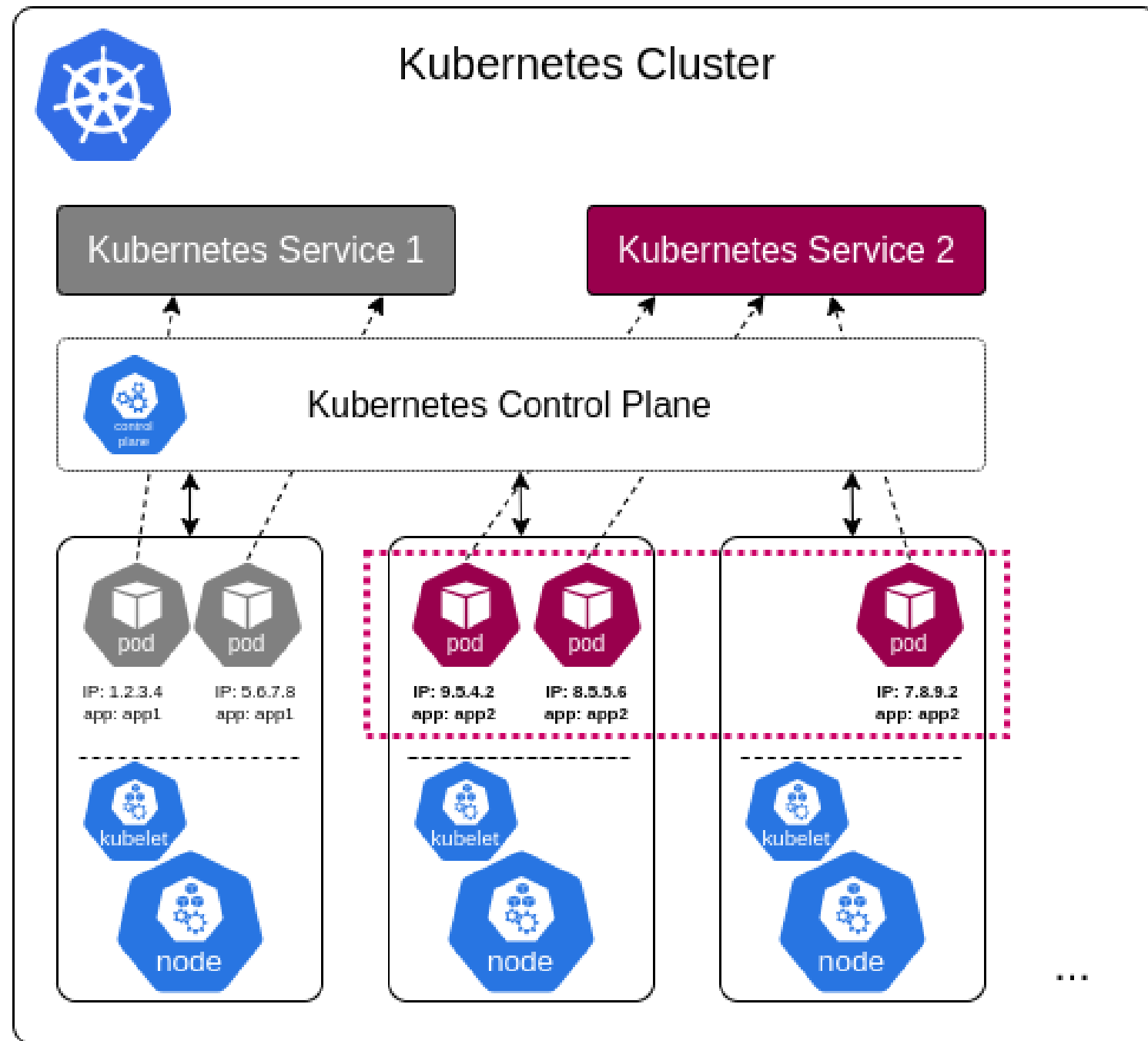
Selectors:

- Can be used to identify objects via labels
- Examples:

```
...
selector:
  environment: prod
  app: my_cool_app
...
```

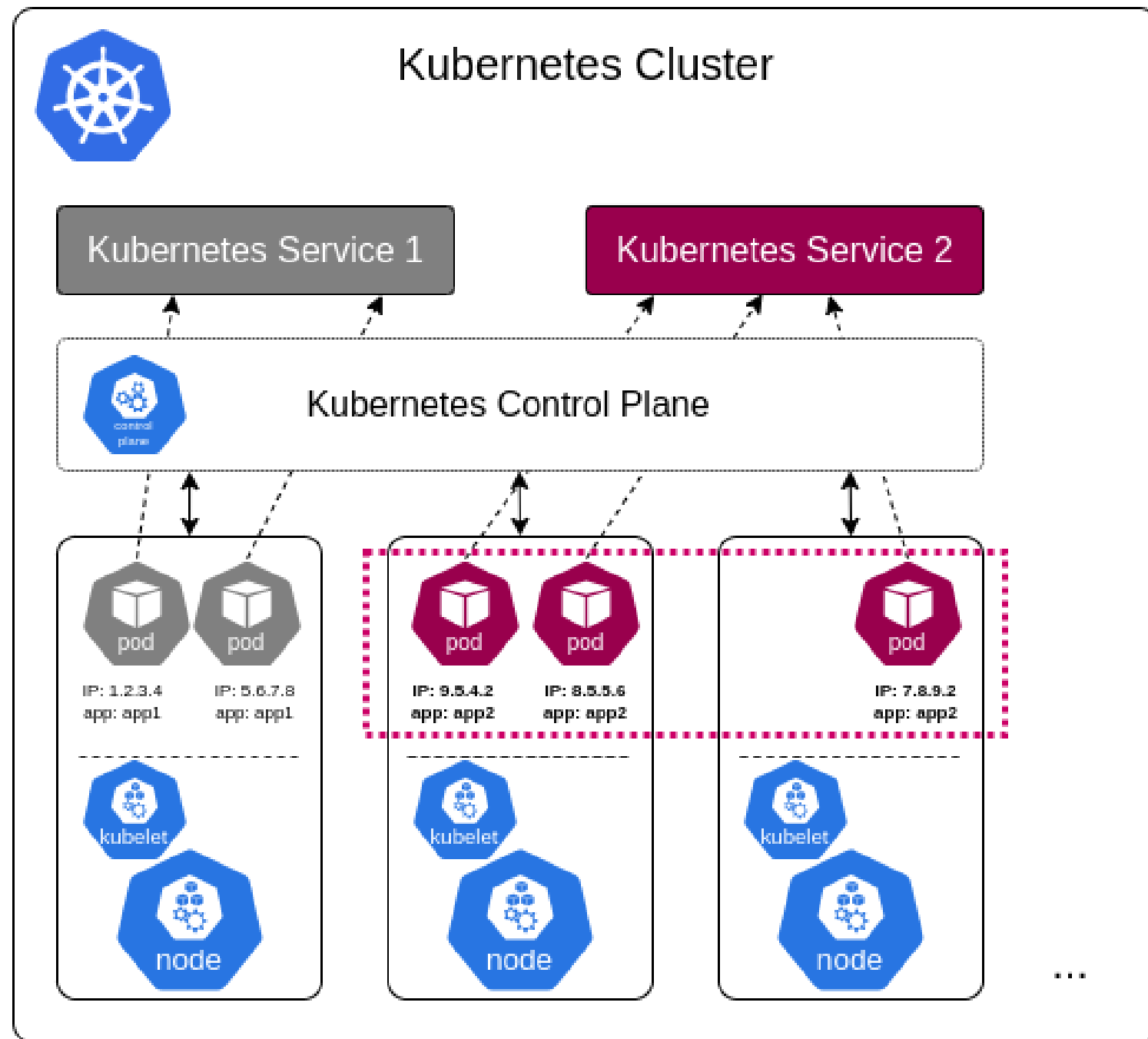
```
...
nodeSelector:
  has_GPU: true
...
```

Networking and Services



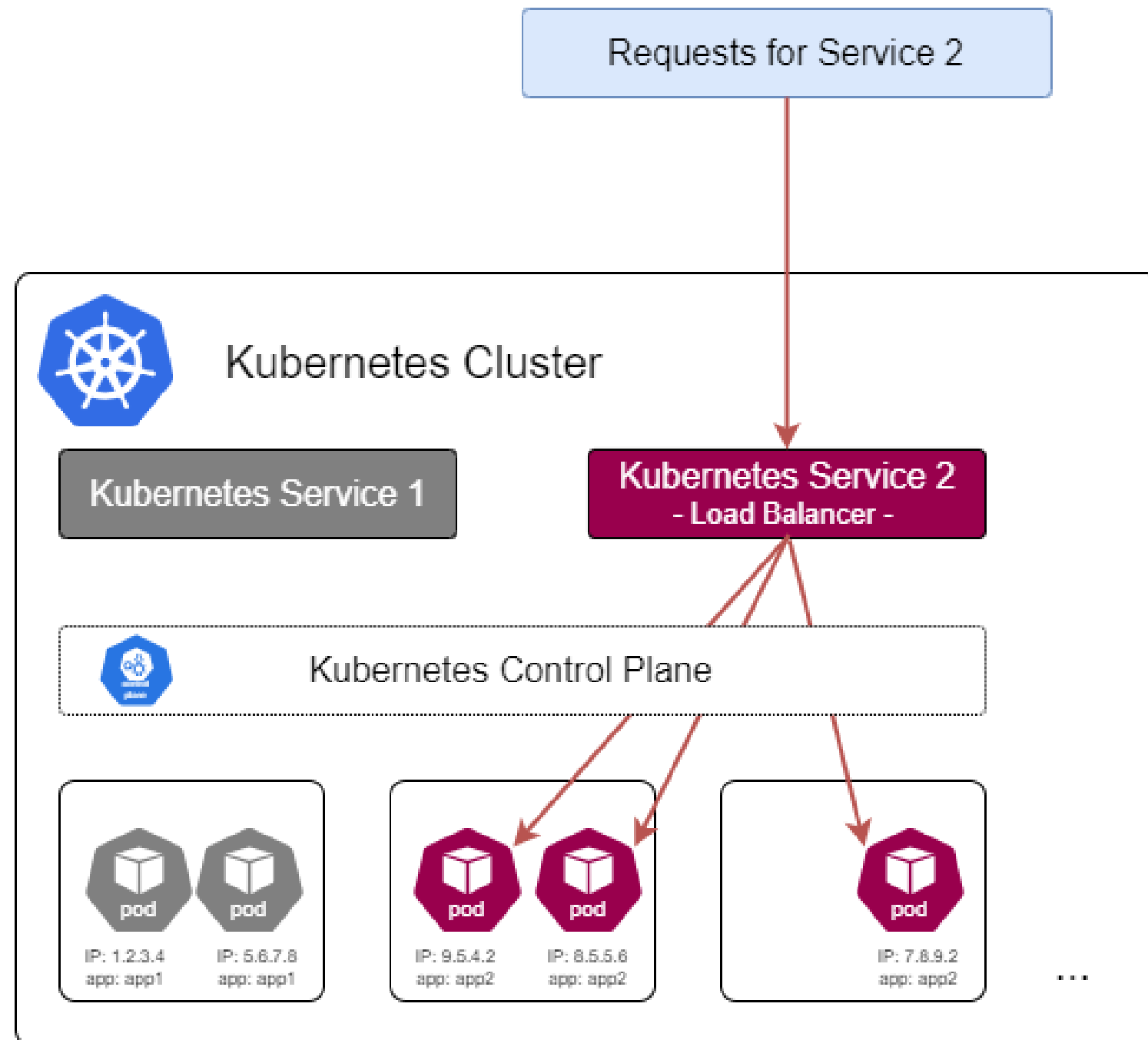
- Each Pod gets its own cluster-wide IP (internet address)
- Can be used for communication between Pods
- Not very useful, as Pods can restart at any time, and will get a new IP
- **Services** are used to attach Pods to, and offer stable connectivity

Service Manifests



```
apiVersion: v1
kind: Service
metadata:
  name: Kubernetes_Service_2
spec:
  type: ...
  selector:
    app: app2
  ...
```

Load Balancing



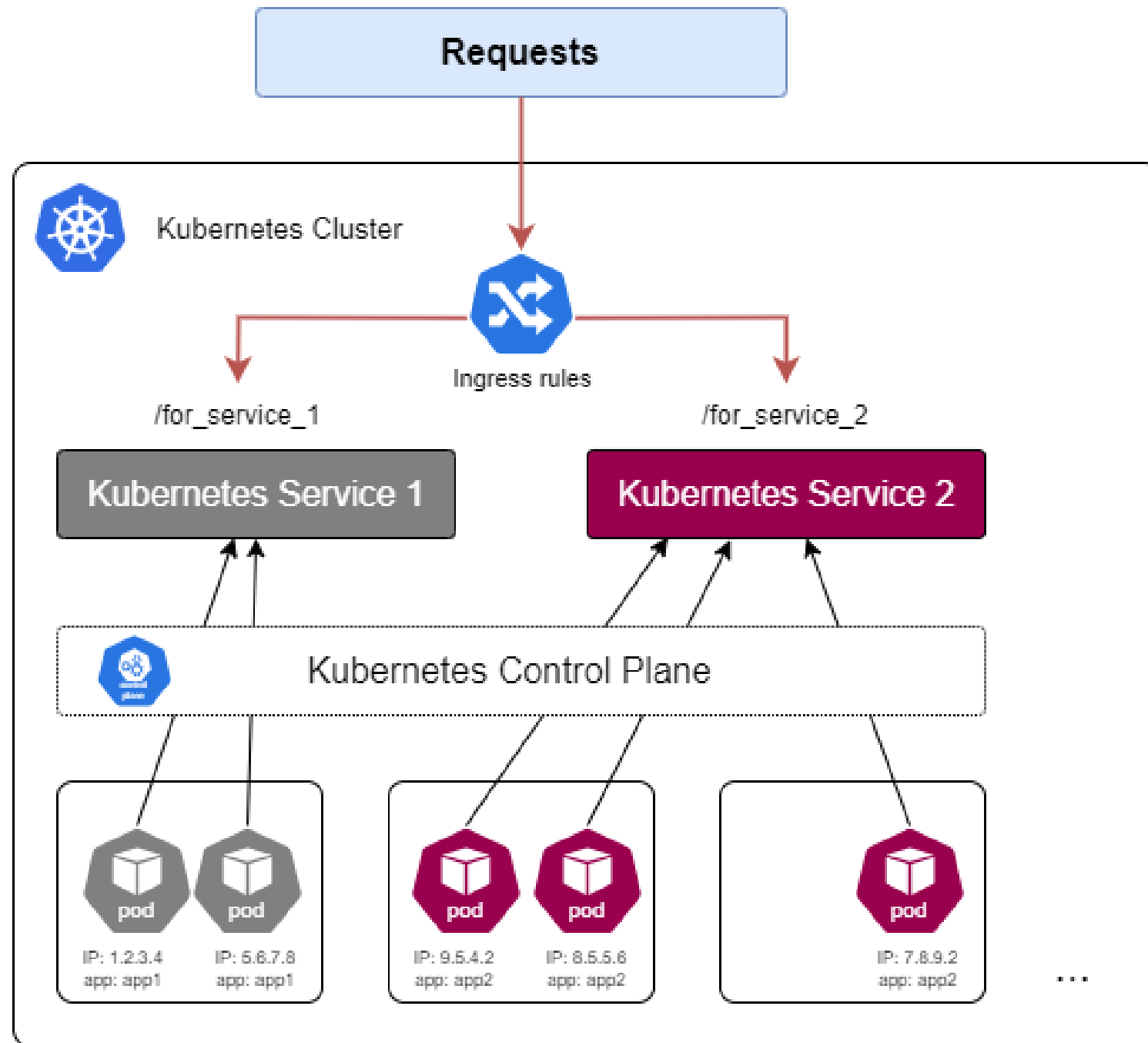
- A **load balancer** in Kubernetes distributes load over Pods
- Avoids uneven load on resources, increases efficiency and lowers response times
- Example:
 - providing a service from multiple Pods
 - load balancer will distribute load evenly to Pods

Load Balancing in Kubernetes

```
apiVersion: v1
kind: Service
metadata:
  name: <service name>
spec:
  type: LoadBalancer
  selector:
    <key1>: <value1>
    <key2>: <value2>
  ...
```

- Load balancers are typically pre-configured by Kubernetes Provider (Cloud Provider)
- No need to declare additional manifests for a load balancer - will automatically be created and attached to the service

Ingress



- Ingress objects are used to route HTTP and HTTPS requests (traffic) from outside the cluster to services in the cluster
- Ingress rules define which requests are served by which service
- Typically used in combination with load balancing

Kubernetes Security

- Security in modern IT architectures is an extremely important, but complex field with many facets
- Kubernetes has all necessary components to secure applications running on it, e.g:
 - the "Secret" API for confidential objects like passwords, tokens, keys etc.
 - tools and APIs to enable encrypted network communication
 - methods for authentication of users
 - *role-based* and *attribute-based access control* ("RBAC" and "ABAC")

Let's practice!
INTRODUCTION TO KUBERNETES

Data Pipelines on Kubernetes

INTRODUCTION TO KUBERNETES

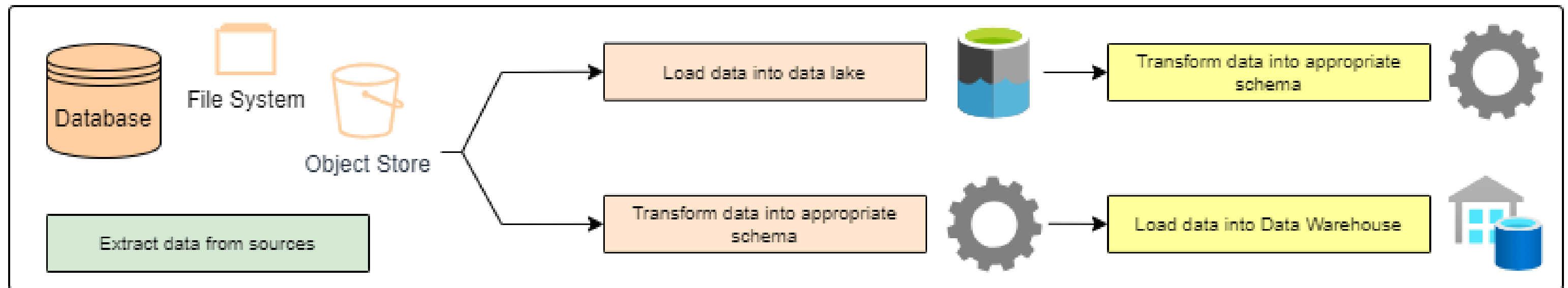


Frank Heilmann

Platform Architect and Freelance
Instructor

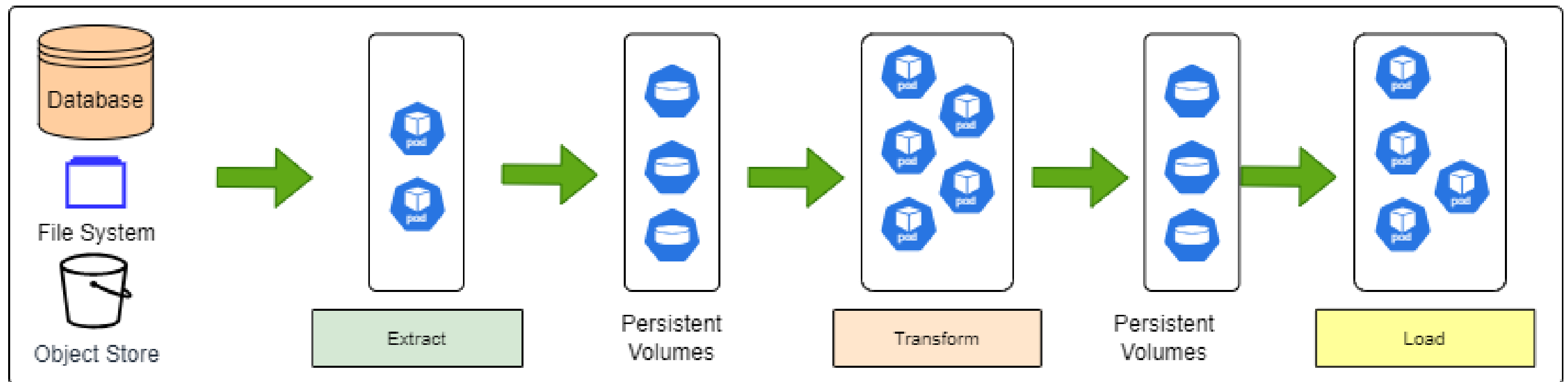
What are Data Pipelines?

- Set of processes to move, transform, or analyze data
- Typical steps:
 - **ETL: Extract** data from various data sources, then **Transform** into a meaningful schema, finally **Load** into a target data sink (e.g., a data warehouse)
 - **ELT: Extract** data from various data sources, then **Load** into a target data sink (e.g., a data lake), finally **Transform** data into meaningful schema when needed



Data Pipelines on Kubernetes

- The steps of a data pipeline map nicely to Kubernetes objects:
 - Extract, Transform, Load steps: Pods (Deployment or StatefulSet)
 - Extracted and Transformed Data: Persistent Volumes
- Kubernetes can scale out Deployments and Storage as required, hence increase throughput



Open-Source Tools for Data Pipelines

- Many open-source software exists that is readily deployable on Kubernetes
- Some examples:
 - Extract: **Apache NiFi, Apache Kafka with Kafka Connect**
 - Transform: **Apache Spark, Apache Kafka, PostgreSQL**
 - Load: **Apache Spark, Apache Kafka with KSQL, PostgreSQL**
 - Storage on top of PVs: **Minio, Ceph**
- This list is by no means complete

Let's practice!
INTRODUCTION TO KUBERNETES

MLOps on Kubernetes

INTRODUCTION TO KUBERNETES



Frank Heilmann

Platform Architect and Freelance
Instructor

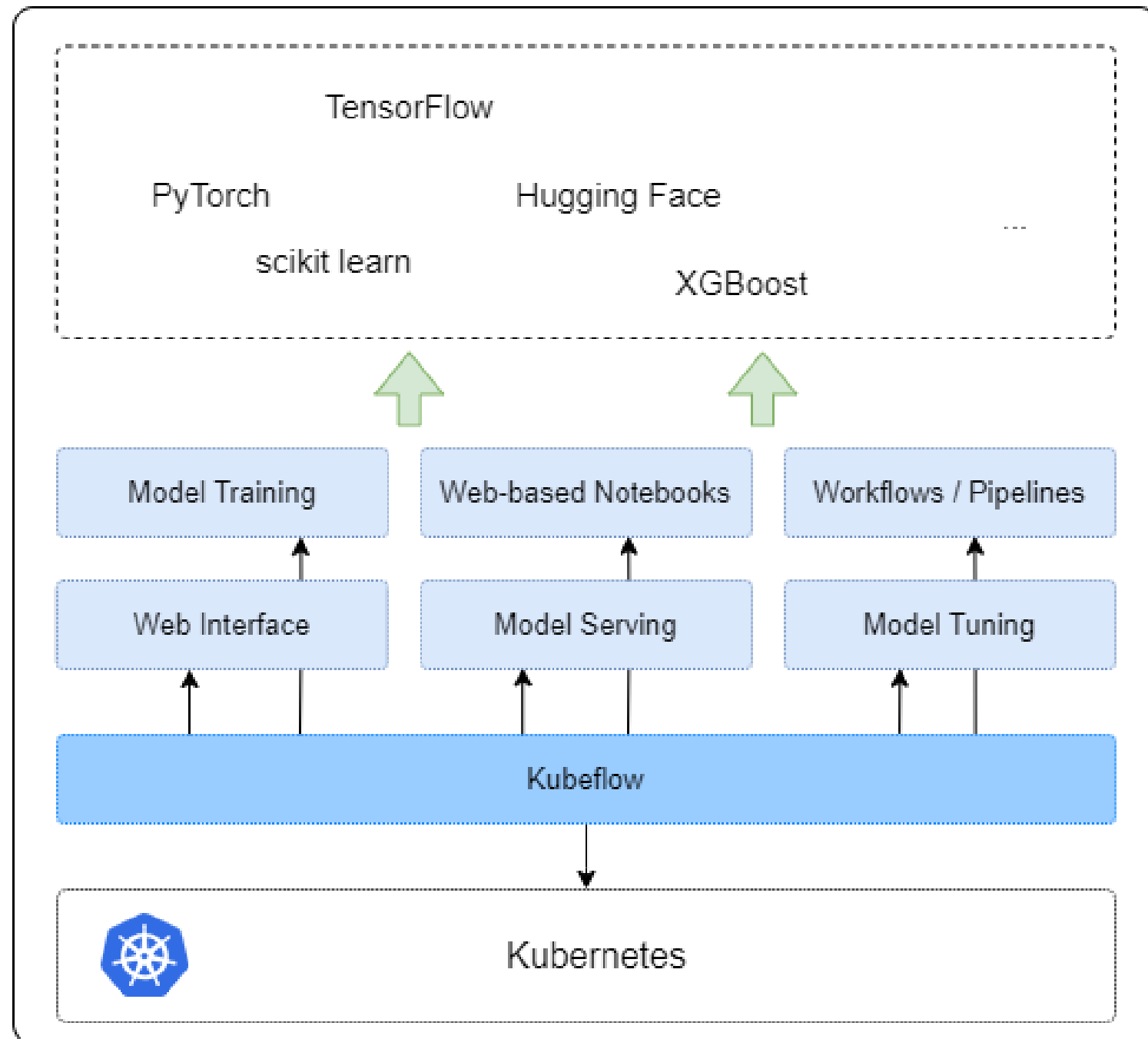
What is MLOps?

- A paradigm to deploy and maintain machine learning models in production
- A set of best-practice workflows with focus on continuous development of such models
- Inspired by *DevOps*:
 - Machine learning models are developed and tested in isolated experimental systems, and then deployed to production
 - When in production, continuous monitoring; retraining may be triggered
- Data scientists, data engineers, and IT teams can work on deployed models synchronously and ensure model accuracy

Implementing MLOps on Kubernetes

- The MLOps paradigm maps very well to Kubernetes:
 - Isolated experimental systems: can be realized via Pods and Kubernetes Storage
 - Monitoring production ML models: enabled via lifecycle of Pods (and deployed image versions)
 - Synchronous work on model accuracy: built in from the very beginning by Kubernetes architecture
- Several frameworks for MLOps exist; the two best-known open-source solutions are
 - [mlflow](#)
 - [Kubeflow](#)

Kubeflow - An Overview



- **Kubeflow** allows simple deployments of ML workflows specifically on Kubernetes
- Covers each step of the ML model lifecycle
- Consists of several components which cover these steps, working independently
- Python can be used to develop and deploy ML models via an API
 - no need to use `kubectl`

Let's practice!
INTRODUCTION TO KUBERNETES

Wrap-Up

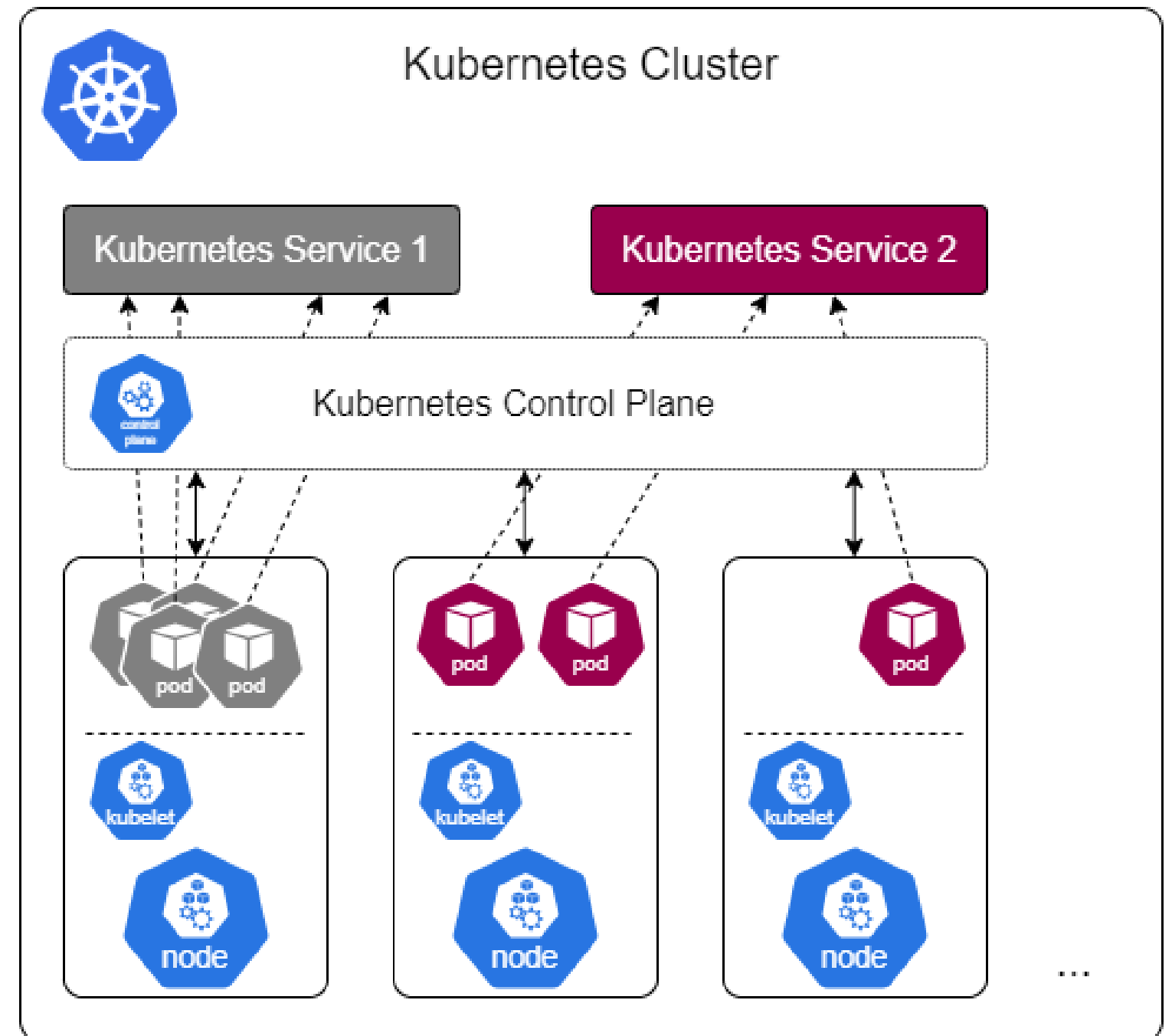
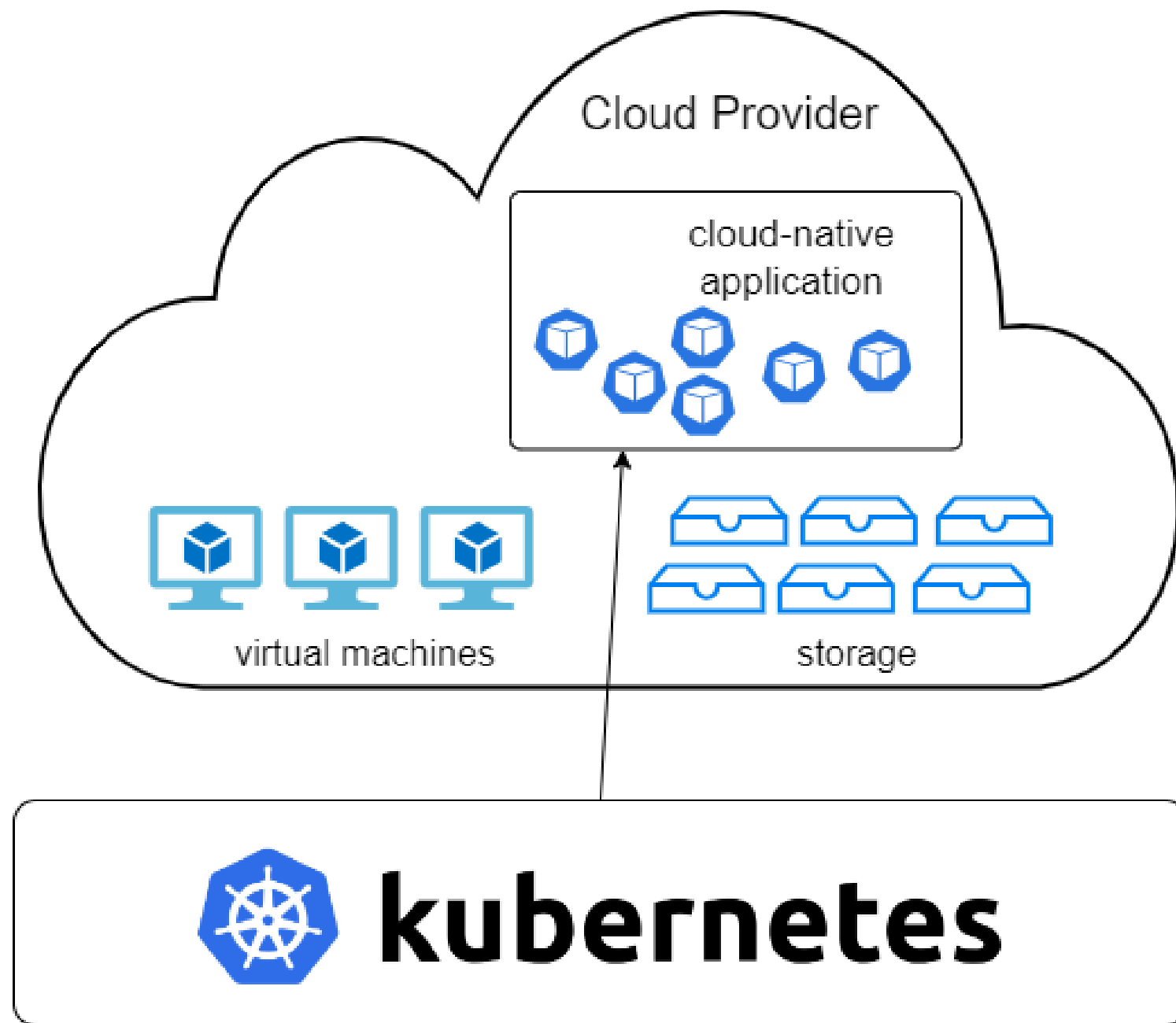
INTRODUCTION TO KUBERNETES



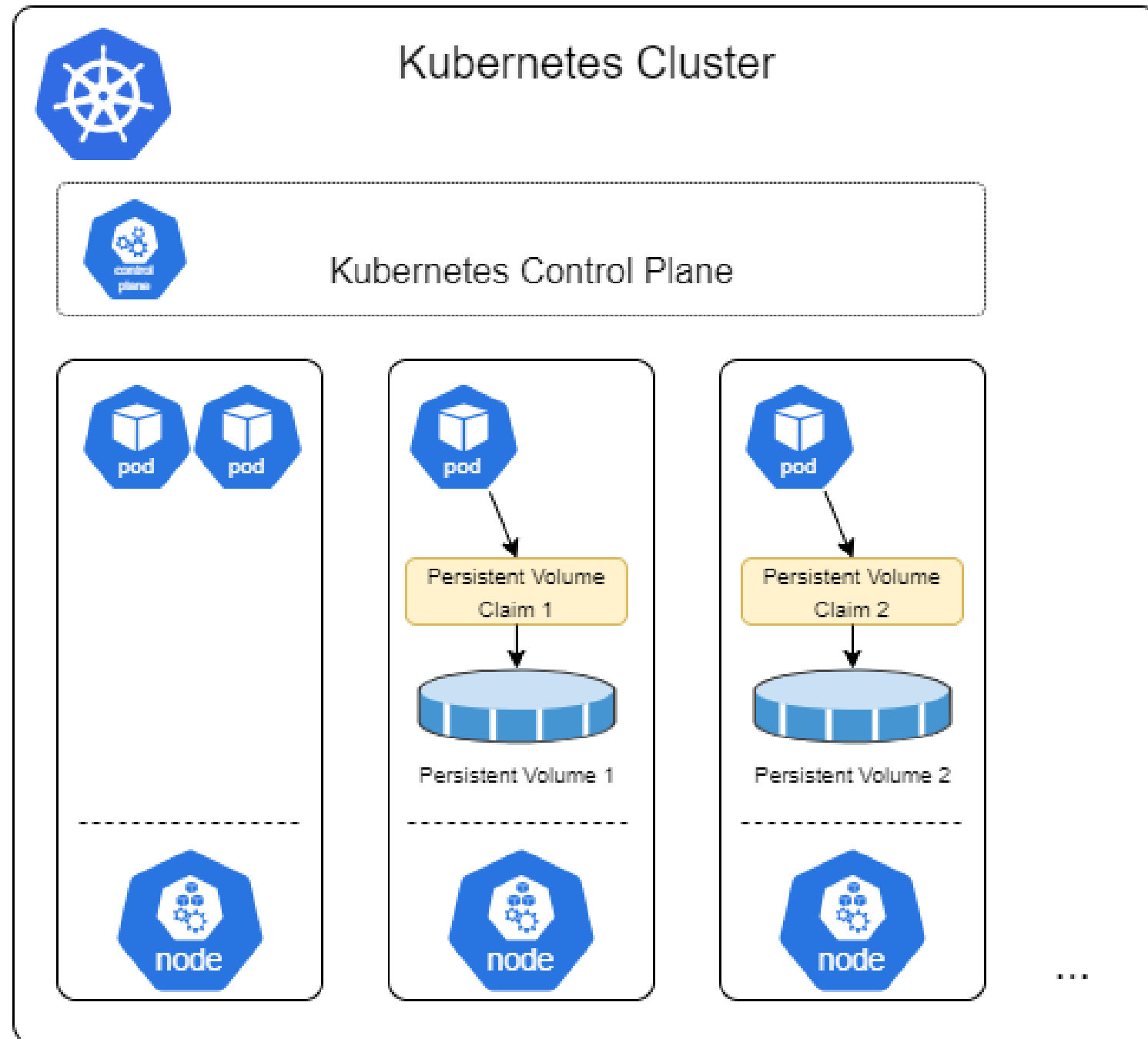
Frank Heilmann

Platform Architect and Freelance
Instructor

What we have learned (1/2)

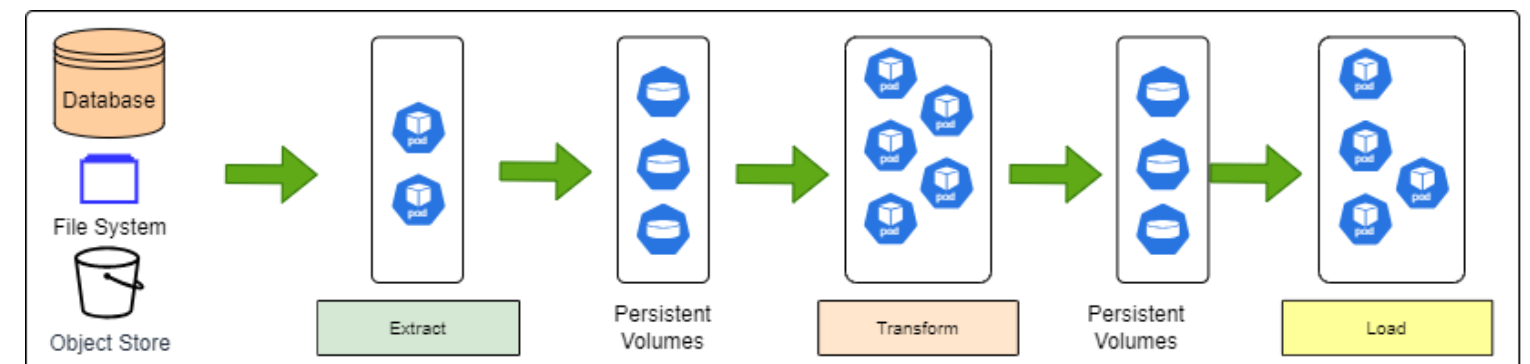
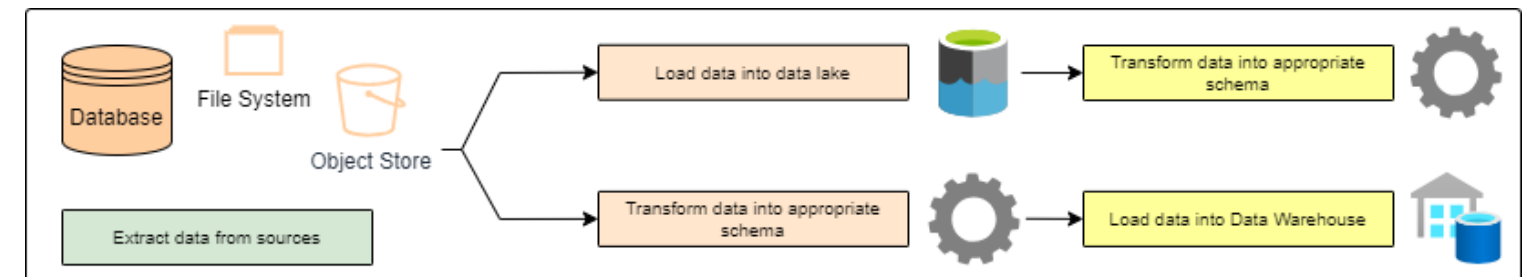


What we have learned (2/2)



```
kubectl apply ...
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  ...
spec:
  ...
```



Where to go from here

- Security for cloud-native applications: **Service Accounts, Secrets, RBAC, etc.**
- Packaging and application lifecycle: **Helm Charts and Kubernetes Operators**
- Advanced storage concepts: **Rook Operators and Ceph**
- CI/CD
- Serverless applications: **KNative**

Thank You!

INTRODUCTION TO KUBERNETES